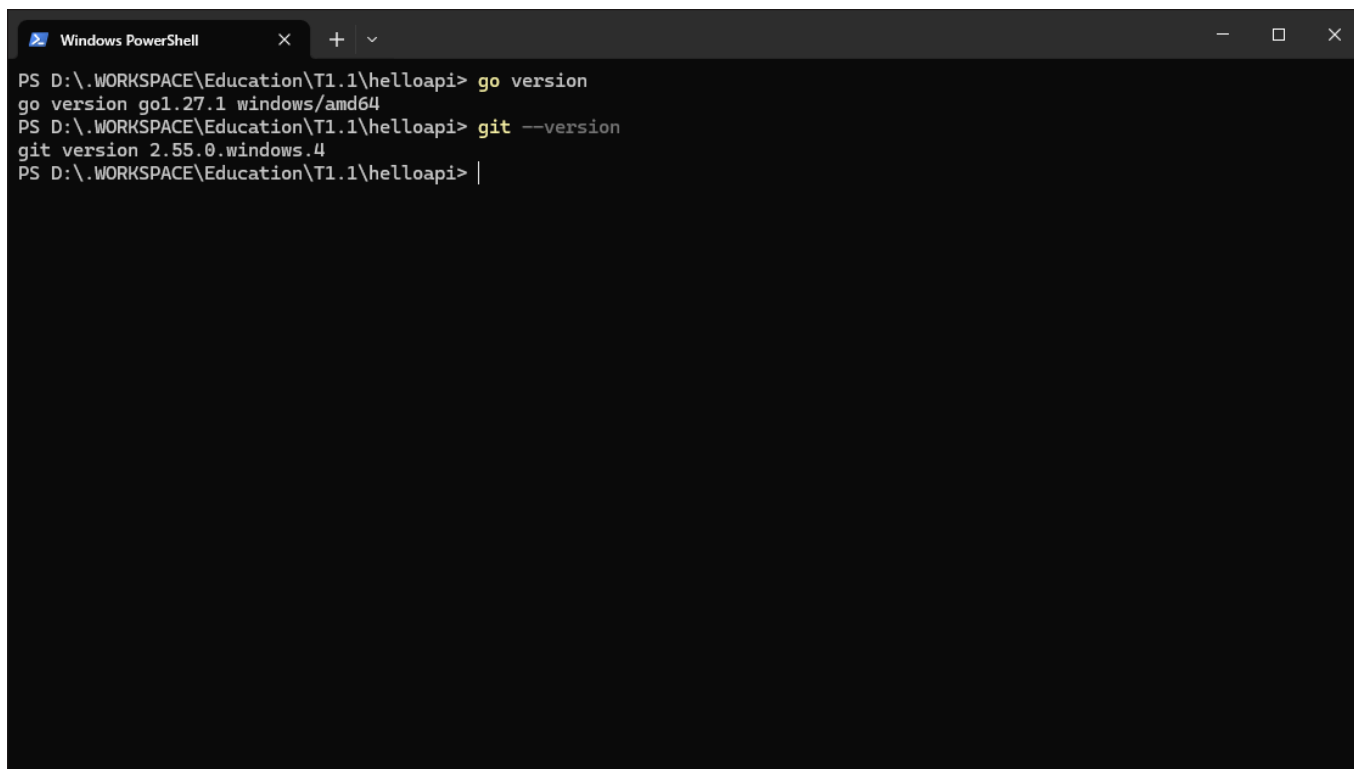


Предварительная подготовка

Убедимся, что установлены необходимые инструменты. Выполним в терминале:

```
go version
git --version
```

A screenshot of a Windows PowerShell terminal window. The title bar shows 'Windows PowerShell' with standard window controls. The terminal content shows the following commands and their outputs:
PS D:\.WORKSPACE\Education\T1.1\helloapi> go version
go version go1.27.1 windows/amd64
PS D:\.WORKSPACE\Education\T1.1\helloapi> git --version
git version 2.55.0.windows.4
PS D:\.WORKSPACE\Education\T1.1\helloapi> |

Создание скелета проекта

Создадим папку проекта, инициализируем Git-репозиторий и Go-модуль.

```
mkdir -p myapp
cd myapp
git init
go mod init github.com/angelomira/myapp
```

```
Windows PowerShell
PS D:\.WORKSPACE\Education\T1.2> mkdir -p myapp

Каталог: D:\.WORKSPACE\Education\T1.2

Mode                LastWriteTime         Length Name
----                -
d-----         25.09.2026   18:28             myapp

PS D:\.WORKSPACE\Education\T1.2> cd myapp
PS D:\.WORKSPACE\Education\T1.2\myapp> git init
Initialized empty Git repository in D:\.WORKSPACE\Education\T1.2\myapp\.git/
PS D:\.WORKSPACE\Education\T1.2\myapp> go mod init github.com/angelomira/myapp
go: creating new go.mod: module github.com/angelomira/myapp
PS D:\.WORKSPACE\Education\T1.2\myapp> |
```

Настроим `.gitignore` для игнора специальных файлов/папок:

```
/bin/
/dist/
/coverage.out
*.log
```

P.S.: при создании репозитории в том же GitHub - можно выбрать пресет для игнорирования файлов, в нашем случае - пресет Go.

Создадим структуру папок:

```
mkdir -p cmd/myapp internal/app utils
```

И создадим файлы:

```
touch cmd/myapp/main.go
touch internal/app/app.go
touch utils/logger.go
```

```
falcion@DESKTOP-KCIB6QV: /  +  v
Mode                LastWriteTime         Length Name
----                -
d-----            25.09.2026      18:28         myapp

PS D:\.WORKSPACE\Education\T1.2> cd myapp
PS D:\.WORKSPACE\Education\T1.2\myapp> git init
Initialized empty Git repository in D:\.WORKSPACE\Education\T1.2\myapp\.git/
PS D:\.WORKSPACE\Education\T1.2\myapp> go mod init github.com/angelomira/myapp
go: creating new go.mod: module github.com/angelomira/myapp
PS D:\.WORKSPACE\Education\T1.2\myapp> bash
falcion@DESKTOP-KCIB6QV:/mnt/d/.WORKSPACE/Education/T1.2/myapp$ ^C
falcion@DESKTOP-KCIB6QV:/mnt/d/.WORKSPACE/Education/T1.2/myapp$ mkdir -p cmd/myapp internal/app utils
falcion@DESKTOP-KCIB6QV:/mnt/d/.WORKSPACE/Education/T1.2/myapp$ touch cmd/myapp/main.go
touch internal/app/app.go
touch utils/logger.go
falcion@DESKTOP-KCIB6QV:/mnt/d/.WORKSPACE/Education/T1.2/myapp$ tree .
.
├── cmd
│   └── myapp
│       └── main.go
├── go.mod
├── internal
│   └── app
│       └── app.go
└── utils
    └── logger.go

6 directories, 4 files
falcion@DESKTOP-KCIB6QV:/mnt/d/.WORKSPACE/Education/T1.2/myapp$ |
```

Создание кода для скелета проекта

Код `cmd/myapp/main.go` файла:

```
package main

import "github.com/angelomira/myapp/internal/app"

func main() {
    app.Run()
}
```

Код `internal/app/app.go` файла:

```
package app

import (
    "encoding/json"
    "fmt"
    "net/http"
    "time"

    "github.com/angelomira/myapp/utils"
)

type pingResp struct {
```

```

        Status string `json:"status"`
        Time    string `json:"time"`
    }

    func Run() {
        mux := http.NewServeMux()

        mux.HandleFunc("/", func(w http.ResponseWriter, r *http.Request) {
            utils.LogRequest(r)
            w.Header().Set("Content-Type", "text/plain; charset=utf-8")
            fmt.Fprintf(w, "Hello, Go project structure!")
        })

        mux.HandleFunc("/ping", func(w http.ResponseWriter, r *http.Request) {
            utils.LogRequest(r)
            w.Header().Set("Content-Type", "application/json; charset=utf-8")

            _ = json.NewEncoder(w).Encode(pingResp{
                Status: "ok",
                Time:    time.Now().UTC().Format(time.RFC3339),
            })
        })

        utils.LogInfo("Server is starting on :8080")
        if err := http.ListenAndServe(":8080", mux); err != nil {
            utils.LogError("server error: " + err.Error())
        }
    }
}

```

Код utils/logger.go файла:

```

package utils

import (
    "fmt"
    "net/http"
    "time"
)

func LogRequest(r *http.Request) {
    fmt.Printf("[%s] %s %s %s\n",
        time.Now().Format(time.RFC3339),
        r.RemoteAddr,
        r.Method,
        r.URL.Path,
    )
}

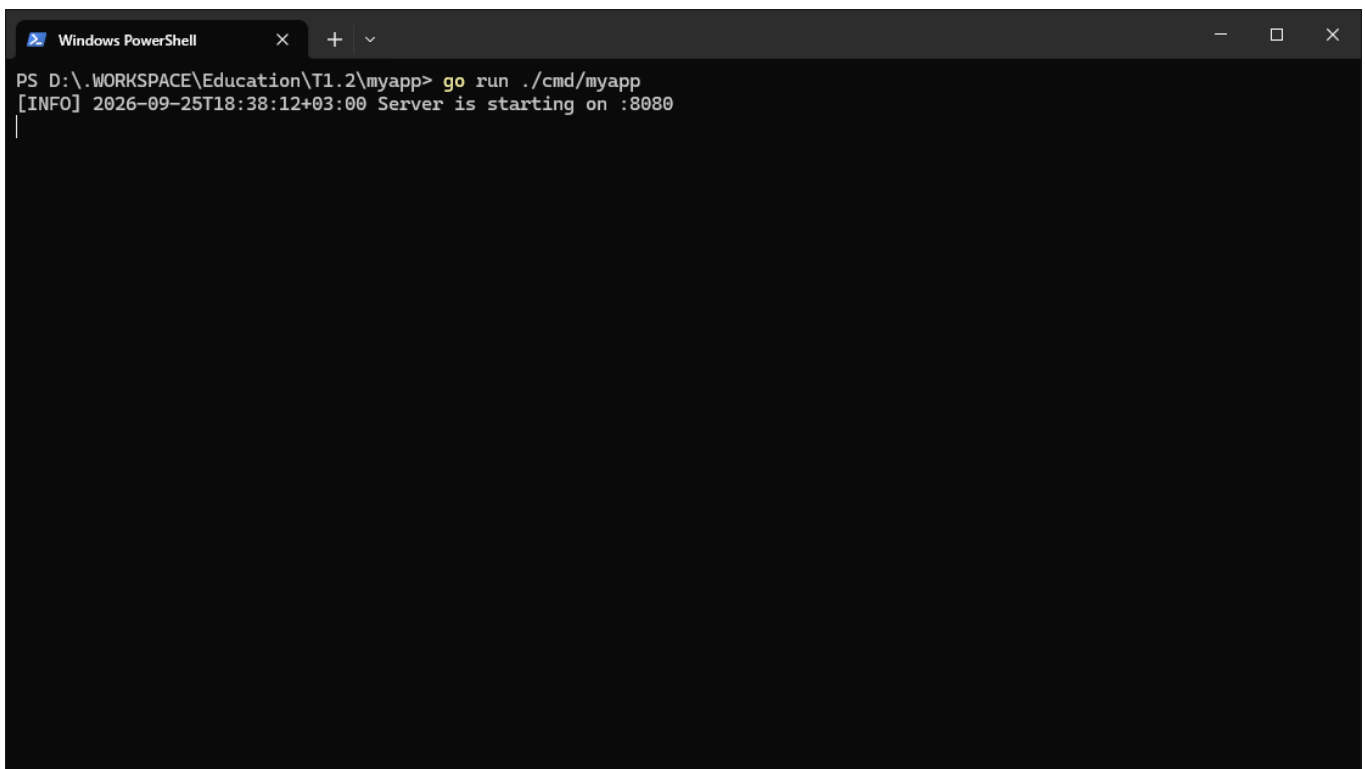
```

```
)  
  
}  
  
func LogInfo(msg string) {  
    fmt.Printf("[INFO] %s %s\n", time.Now().Format(time.RFC3339), msg)  
}  
  
func LogError(msg string) {  
    fmt.Printf("[ERROR] %s %s\n", time.Now().Format(time.RFC3339), msg)  
}
```

Запуск и быстрая проверка

Запустим сервер:

```
go run ./cmd/myapp
```

A screenshot of a Windows PowerShell terminal window. The title bar shows 'Windows PowerShell' with standard window controls. The command prompt shows the current directory as 'D:\.WORKSPACE\Education\T1.2\myapp'. The user has entered the command 'go run ./cmd/myapp'. The output shows an information message: '[INFO] 2026-09-25T18:38:12+03:00 Server is starting on :8080'. The terminal background is black, and the text is white.

```
PS D:\.WORKSPACE\Education\T1.2\myapp> go run ./cmd/myapp  
[INFO] 2026-09-25T18:38:12+03:00 Server is starting on :8080  
|
```

Проверим эндпоинты:

```
curl http://localhost:8080/  
curl http://localhost:8080/ping
```

```
Windows PowerShell x Windows PowerShell + v
PS C:\Users\Falcion> curl http://localhost:8080/

StatusCode      : 200
StatusDescription : OK
Content         : Hello, Go project structure!
RawContent      : HTTP/1.1 200 OK
                  Content-Length: 28
                  Content-Type: text/plain; charset=utf-8
                  Date: Fri, 25 Sep 2026 15:39:48 GMT

                  Hello, Go project structure!
Forms           : {}
Headers         : {[Content-Length, 28], [Content-Type, text/plain; charset=utf-8], [Date, Fri, 25 Sep 2026 15:39:48 GMT]}
Images          : {}
InputFields     : {}
Links           : {}
ParsedHtml      : mshtml.HTMLDocumentClass
RawContentLength : 28

PS C:\Users\Falcion> curl http://localhost:8080/ping

StatusCode      : 200
StatusDescription : OK
Content         : {"status":"ok","time":"2026-09-25T15:39:50Z"}
```

```
Windows PowerShell x Windows PowerShell + v
RawContentLength : 28

PS C:\Users\Falcion> curl http://localhost:8080/ping

StatusCode      : 200
StatusDescription : OK
Content         : {"status":"ok","time":"2026-09-25T15:39:50Z"}
RawContent      : HTTP/1.1 200 OK
                  Content-Length: 46
                  Content-Type: application/json; charset=utf-8
                  Date: Fri, 25 Sep 2026 15:39:50 GMT

                  {"status":"ok","time":"2026-09-25T15:39:50Z"}
Forms           : {}
Headers         : {[Content-Length, 46], [Content-Type, application/json; charset=utf-8], [Date, Fri, 25 Sep 2026 15:39:50 GMT]}
Images          : {}
InputFields     : {}
Links           : {}
ParsedHtml      : mshtml.HTMLDocumentClass
RawContentLength : 46

PS C:\Users\Falcion> |
```

Сборка бинарника

Соберем проект и проверим его:

```
go build -o bin/myapp.exe ./cmd/myapp
./bin/myapp.exe
```

```
Windows PowerShell X Windows PowerShell X + v
PS D:\.WORKSPACE\Education\T1.2\myapp> go build -o bin/myapp.exe ./cmd/myapp
PS D:\.WORKSPACE\Education\T1.2\myapp> ./bin/myapp.exe
[INFO] 2026-09-25T18:42:06+03:00 Server is starting on :8080
```

Дополнительные задания

Задание А: Request-ID и улучшение логов

Добавим генератор ID в `utils/logger.go`:

```
import (
    "crypto/rand"
    "encoding/hex"
    // ...
)

func NewID16() string {
    b := make([]byte, 8)
    _, _ = rand.Read(b)
    return hex.EncodeToString(b)
}
```

Добавим прослойку в `internal/app/app.go`:

```
func withRequestID(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        id := r.Header.Get("X-Request-Id")
        if id == "" {
            id = utils.NewID16()
        }
        w.Header().Set("X-Request-Id", id)
        next.ServeHTTP(w, r)
    })
}
```

```

    }
    w.Header().Set("X-Request-Id", id)
    next.ServeHTTP(w, r)
  })
}

```

И применим эту прослойку при запуске сервера:

```

// ...
handler := withRequestID(mux)
utils.LogInfo("Server is starting on :8080")
if err := http.ListenAndServe(":8080", handler); err != nil {
    utils.LogError("server error: " + err.Error())
}

```

Проверим:

```

curl.exe http://localhost:8080/ping
curl.exe -H "X-Request-Id: demo-123" http://localhost:8080/ping

```

The screenshot shows a Windows PowerShell window with two tabs. The active tab shows the following commands and output:

```

PS C:\Users\Falcion> curl.exe http://localhost:8080/ping
{"status":"ok","time":"2026-09-25T15:48:08Z"}
PS C:\Users\Falcion> curl.exe -H "X-Request-Id: demo-123" http://localhost:8080/ping
{"status":"ok","time":"2026-09-25T15:48:18Z"}
PS C:\Users\Falcion> |

```

P.S. используем именно `curl.exe` т.к. `curl` является псевдонимом `Invoke-WebRequest`

Задание В: единый формат JSON-ошибок

Создадим `utils/httpjson.go`:

```
package utils

import (
    "encoding/json"
    "net/http"
)

type JSONError struct {
    Error string `json:"error"`
}

func WriteJSON(w http.ResponseWriter, code int, v any) {
    w.Header().Set("Content-Type", "application/json; charset=utf-8")
    w.WriteHeader(code)
    _ = json.NewEncoder(w).Encode(v)
}

func WriteErr(w http.ResponseWriter, code int, msg string) {
    WriteJSON(w, code, JSONError{Error: msg})
}
```

Добавим маршрут `/fail` в `app.go`:

```
mux.HandleFunc("/fail", func(w http.ResponseWriter, r *http.Request) {
    utils.LogRequest(r)
    utils.WriteErr(w, http.StatusBadRequest,
"bad_request_example")
})
```

Проверим:

```
{ "status": "OK", "time": "2020-09-20T10:40:10Z" }
PS C:\Users\Falcion> curl.exe -i http://localhost:8080/fail
HTTP/1.1 200 OK
Content-Type: text/plain; charset=utf-8
X-Request-Id: 7f138037008171fa
Date: Fri, 25 Sep 2026 15:51:15 GMT
Content-Length: 28

Hello, Go project structure!
```

Задание С: разделение обработчиков

Создадим папку `internal/app/handlers` и файл `ping.go`:

```
mkdir internal/app/handlers
touch internal/app/handlers/ping.go
```

```
package handlers

import (
    "encoding/json"
    "net/http"
    "time"

    "github.com/angelomira/myapp/utils"
)

type pingResp struct {
    Status string `json:"status"`
    Time   string `json:"time"`
}

func Ping(w http.ResponseWriter, r *http.Request) {
    utils.LogRequest(r)
    w.Header().Set("Content-Type", "application/json; charset=utf-8")
    _ = json.NewEncoder(w).Encode(pingResp{
        Status: "ok",
        Time:   time.Now().UTC().Format(time.RFC3339),
    })
}
```

В `app.go` подключим пакет и заменим анонимную функцию на `handlers.Ping`:

```
import (
    // ...
    "github.com/angelomira/myapp/internal/app/handlers"
)
// ...
mux.HandleFunc("/ping", handlers.Ping)
```

```
logout
PS C:\Users\Falcion> curl.exe -i http://localhost:8080/ping
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
X-Request-Id: cab1f35b793b832a
Date: Fri, 25 Sep 2026 15:55:53 GMT
Content-Length: 46

{"status":"ok","time":"2026-09-25T15:55:53Z"}
PS C:\Users\Falcion> |
```

Коммит на репозиторию

P.S. в моём случае я коммичу проект на одну из веток репозиторий, но логика одна и та же.

Закоммитим проект и т.п. под собственными условиями:

```
git checkout
git add .
git commit -m "Add practice 02 to the workspace"
git push
```

Эссе и документация проекта прилагается в начале это файла на репозитории.